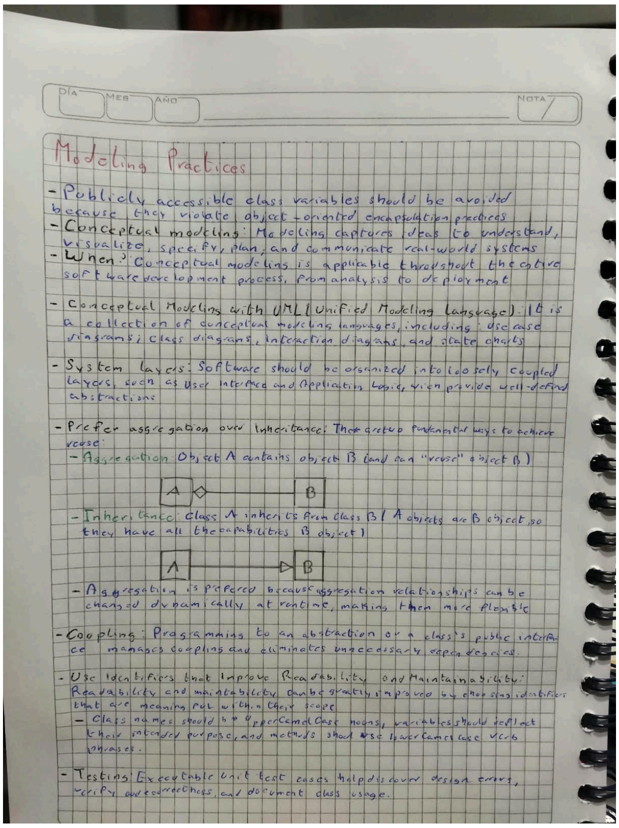


DÍA MES AÑO NOTA /

Modeling Practices

- Publicly accessible class variables should be avoided because they violate object-oriented encapsulation practices
- Conceptual modeling: Modeling captures ideas to understand, visualize, specify, plan, and communicate real-world systems
- When? Conceptual modeling is applicable throughout the entire software development process, from analysis to deployment

- Conceptual Modeling with UML (Unified Modeling Language): It is a collection of conceptual modeling languages, including: use case diagrams; class diagrams, interaction diagrams, and state charts

- System layers: Software should be organized into loosely coupled layers, such as User Interface and Application Logic, which provide well-defined abstractions

- Prefer aggregation over inheritance: There are two fundamental ways to achieve reuse:
  - Aggregation: Object A contains object B (and can "reuse" object B)

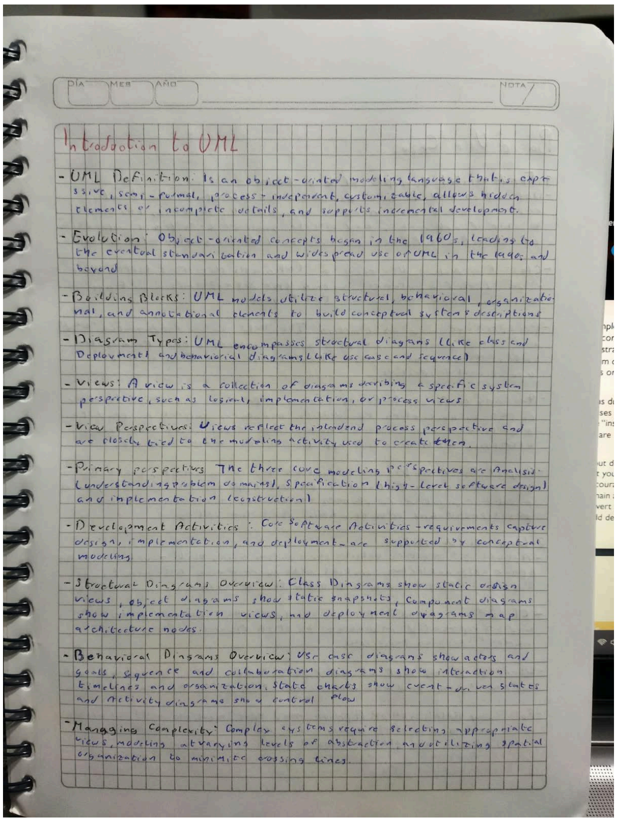
  - Inheritance: Class A inherits from Class B (A objects are B object, so they have all the capabilities B object)

  - Aggregation is preferred because aggregation relationships can be changed dynamically at runtime, making them more flexible

- Coupling: Programming to an abstraction or a class's public interface manages coupling and eliminates unnecessary dependencies.

- Use identifiers that improve Readability and Maintainability: Readability and maintability can be greatly improved by choosing identifiers that are meaningful within their scope
  - Class names should be UpperCamelCase nouns, variables should reflect their intended purpose, and methods should use lowerCamelCase verb phrases.

- Testing: Executable unit test cases help discover design errors, verify and correctness, and document class usage.



DÍA MES AÑO NOTA /

Introduction to UML

- UML Definition: Is an object-oriented modeling language that is expressive, semi-formal, process-independent, customizable, allows hidden elements or incomplete details, and supports incremental development.

- Evolution: Object-oriented concepts began in the 1960s, leading to the eventual standarization and widespread use of UML in the 1990s and beyond

- Building Blocks: UML models utilize structural, behavioral, organizational, and annotational elements to build conceptual systems descriptions

- Diagram Types: UML encompasses structural diagrams (like class and Deployment) and behaviorial diagrams (like use case and sequence)

- Views: A view is a collection of diagrams describing a specific system perspective, such as logical, implementation, or process views

- View Perspectives: Views reflect the intended process perspective and are closely tied to the modeling activity used to create them.

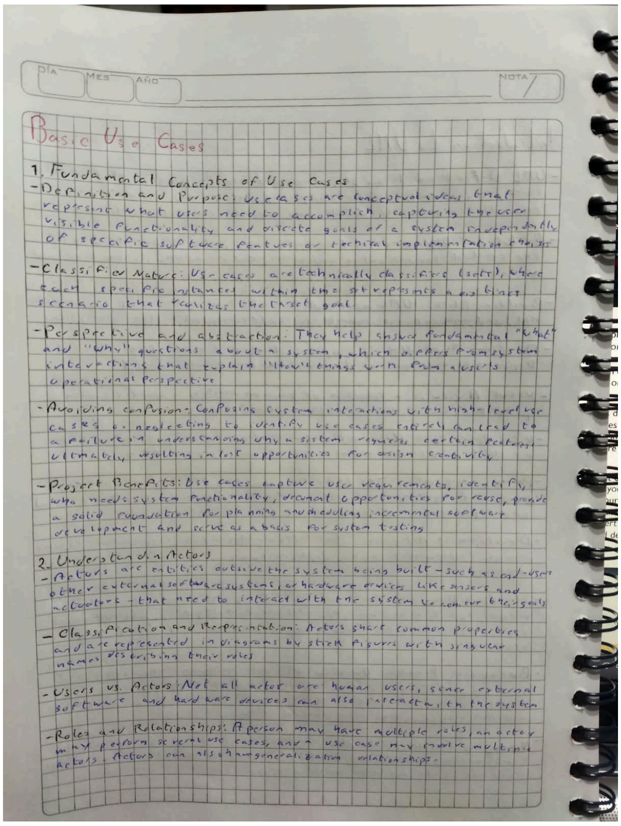
- Primary perspectives: The three core modeling perspectives are Analysis (understanding problem domains), Specification (high-level software design) and implementation (construction)

- Development Activities: Core Software Activities - requirements capture design, implementation, and deployment - are supported by conceptual modeling

- Structural Diagrams Overview: Class Diagrams show static design views, object diagrams show static snapshots, Component diagrams show implementation views, and deployment diagrams map architecture nodes.

- Behavioral Diagrams Overview: Use case diagrams show actors and goals, sequence and collaboration diagrams show interaction timelines and organization, State charts show event-driven states and Activity diagrams show control flow

- Managing Complexity: Complex systems require selecting appropriate views, modeling at varying levels of abstraction and utilizing spatial organization to minimize crossing lines.



DÍA MES AÑO NOTA /

Basic Use Cases

1. Fundamental Concepts of Use Cases

- Definition and Purpose: Use cases are conceptual ideas that represent what users need to accomplish, capturing the user visible functionality and discrete goals of a system independently of specific software features or technical implementation choices

- Classifier Nature: Use cases are technically classifiers (sets), where each specific instances within the set represents a distinct scenario that realizes the target goal

- Perspective and abstraction: They help answer fundamental "what" and "why" questions about a system, which differs from system interactions that explain "How" things work from a user's operational perspective

- Avoiding confusion: Confusing system interactions with high-level use cases or neglecting to identify use cases entirely can lead to a failure in understanding why a system requires certain features, ultimately resulting in lost opportunities for design creativity

- Project Benefits: Use cases capture user requirements, identify who needs system functionality, document opportunities for reuse, provide a solid foundation for planning and scheduling incremental software development and serve as a basis for system testing

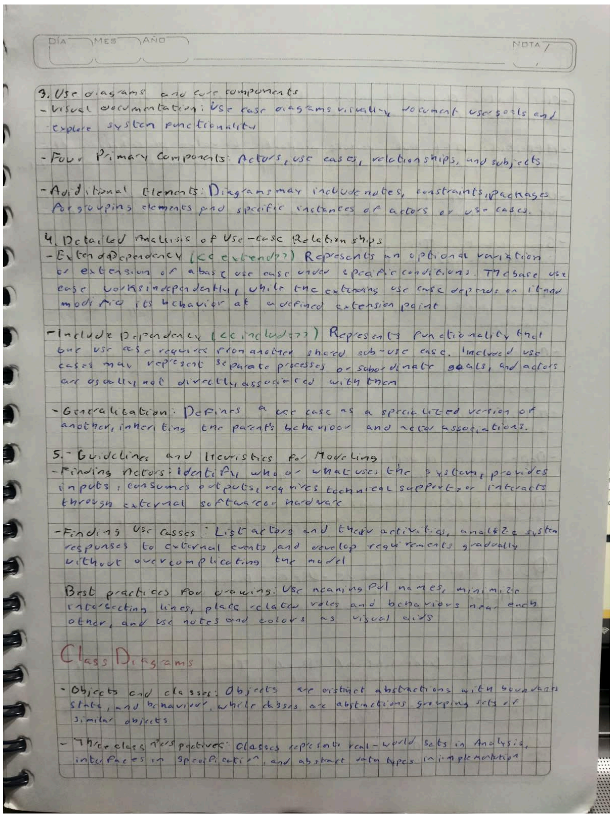
2. Understanding Actors

- Actors are entities outside the system being built - such as end-users other external software systems, or hardware devices like sensors and actuators - that need to interact with the system to achieve their goals

- Classification and Representation: Actors share common properties and are represented in diagrams by stick figures with singular names describing their roles

- Users vs. Actors: Not all actor are human users, since external software and hardware devices can also interact with the system

- Roles and Relationships: A person may have multiple roles, an actor may perform several use cases, and a use case may involve multiple actors. Actors can also have generalization relationships.



DÍA MES AÑO NOTA /

3. Use diagrams and core components

- Visual documentation: Use case diagrams visually document user goals and explore system functionality

- Four Primary Components: Actors, use cases, relationships, and subjects

- Additional Elements: Diagrams may include notes, constraints, packages for grouping elements and specific instances of actors or use cases.

4. Detailed Analysis of Use-case Relationships

- Extend Dependency (<<extend>>) Represents an optional variation or extension of a base use case under specific conditions. The base use case works independently, while the extending use case depends on it and modifies its behavior at a defined extension point

- Include Dependency (<<include>>) Represents functionality that one use case requires from another shared sub-use case. Included use cases may represent separate processes or subordinate goals, and actors are usually not directly associated with them

- Generalization: Defines a use case as a specialized version of another, inheriting the parent's behaviour and actor associations.

5.- Guidelines and Heuristics for Modeling

- Finding Actors: Identify who or what uses the system, provides inputs, consumes outputs, requires technical support, or interacts through external software or hardware

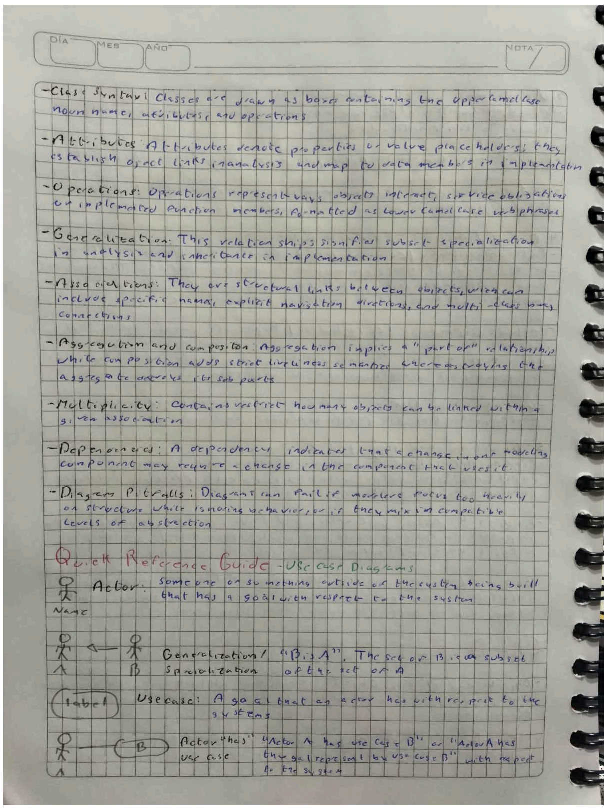
- Finding Use cases: List actors and their activities, analyze system responses to external events and develop requirements gradually without overcomplicating the model

Best practices for drawing: Use meaningful names, minimize intersecting lines, place related roles and behaviors near each other, and use notes and colors as visual aids

Class Diagrams

- Objects and classes: Objects are distinct abstractions with boundaries state, and behavior, while classes are abstractions grouping sets of similar objects

- Three class Perspectives: Classes represent real-world sets in Analysis, interfaces in Specification, and abstract data types in implementation


DÍA
MES
AÑO
NOTA
Name
A
B
label
B
A

- Class Syntax: Classes are drawn as boxes containing the UpperCamelCase noun name, attributes, and operations

- Attributes: Attributes denote properties or value placeholders; they establish object links in analysis and map to data members in implementation

- Operations: Operations represent ways objects interact, service obligations or implemented function members, formatted as lowerCamelCase verb phrases

- Generalization: This relationships signifies subset specialization in analysis and inheritance in implementation

- Associations: They are structural links between objects, which can include specific names, explicit navigation directions, and multi-class n-ary connections

- Aggregation and composition: Aggregation implies a "part of" relationship while composition adds strict liveliness semantics where destroying the aggregate destroys its sub parts

- Multiplicity: Constrains restrict how many objects can be linked within a given association

- Dependencies: A dependency indicates that a change in one modeling component may require a change in the component that uses it.

- Diagram Pitfalls: Diagrams can fail if modelers focus too heavily on structure while ignoring behavior, or if they mix incompatible levels of abstraction

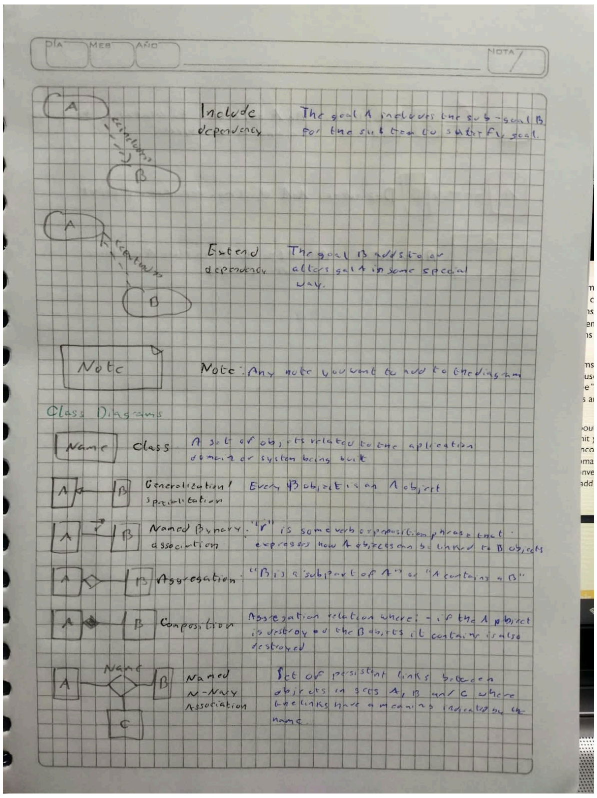
Quick Reference Guide - Use case Diagrams

Actor: Someone or something outside of the system being built that has a goal with respect to the system

Generalization/Specialization: "B is A". The set of B is a subset of the set of A

Use case: A goal that an actor has with respect to the systems

Actor "has" use case: "Actor A has use case B" or "Actor A has the goal represent by use case B" with respect to the system


DÍA
MES
AÑO
NOTA
A
«include»
B
A
«extend»
B
Note
Name
A
B
A
r
B
A
B
A
B
Name
A
B
C

Include dependency: The goal A includes the sub-goal B for the subitem to satisfy goal.

Extend dependency: The goal B adds to or alters goal A in some special way.

Note: Any note you want to add to the diagram

Class Diagrams

Class: A set of objects related to the aplication domain or system being built

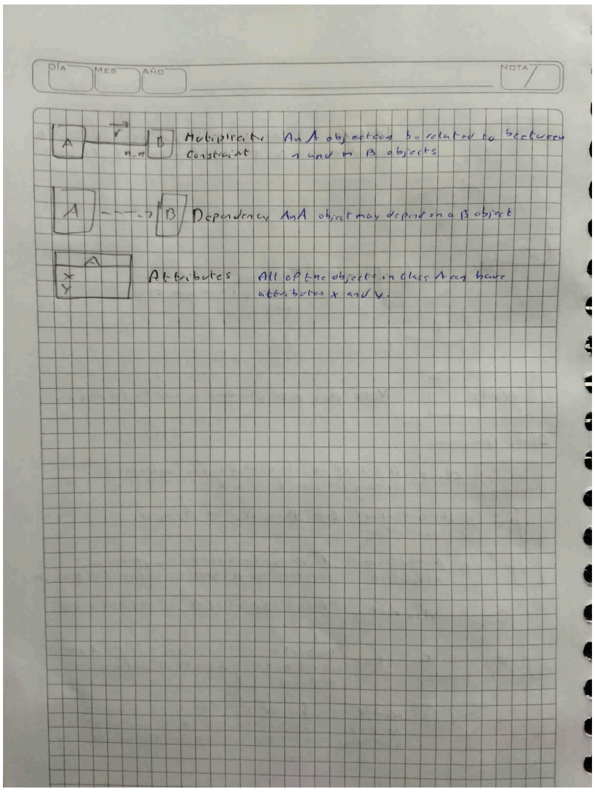
Generalization/Specialization: Every B object is an A object

Named Binary association: "r" is some verb or preposition phrase that expresses how A objects can be linked to B objects

Aggregation: "B is a subpart of A" or "A contains a B"

Composition: Aggregation relation where: - if the A object is destroyed the B objects it contains is also destroyed

Named N-Nary Association: Set of persistent links between objects in sets A, B and/C where the links have a meaning indicated by the name.


DÍA
MES
AÑO
NOTA
A
r
n..m
B
A
B
A
x
Y

Multiplicity Constraint: An A object can be related to between n and m B objects

Dependency: An A object may depend on a B object

Attributes: All of the objects in class A can have attributes x and y.